



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ИНФОРМАЦИОННЫЕ СИСТЕМЫ В ХТ

Лекция 27. Оркестровка и кластеризация

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

 Задачи, требующие оркестровки приложений и кластеризации

 Docker Swarm

- ◉ Создание кластера
- ◉ Узлы в кластере
- ◉ Сервисы
- ◉ Масштабирование сервисов
- ◉ Распределение нагрузки

 Kubernetes

- ◉ Задачи, архитектура и основные сущности
- ◉ Декларативный и императивные подходы в управлении
- ◉ Сервисы
- ◉ Deployment и ReplicaSet
- ◉ Компоненты Kubernetes

Микросервисная архитектура

● Микросервисная архитектура приложения

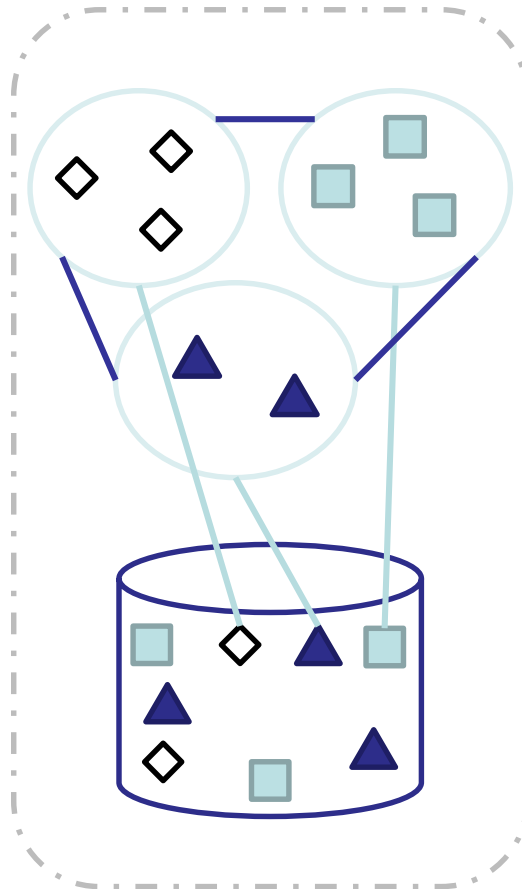
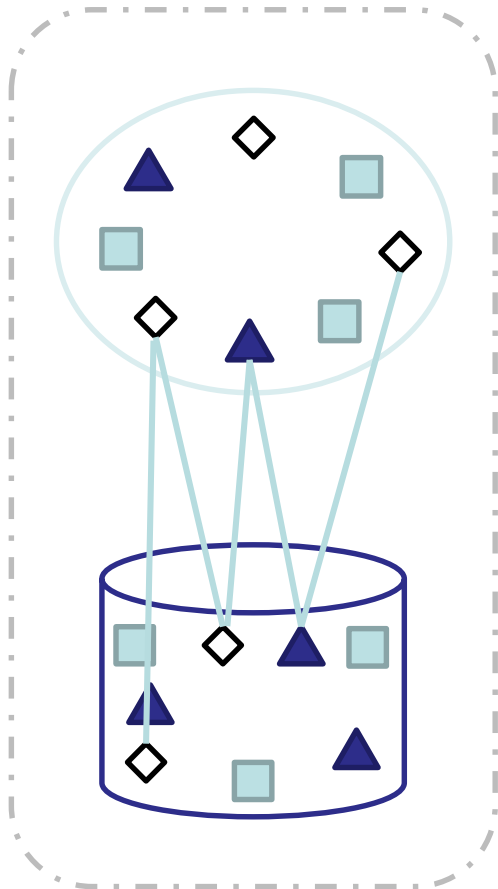
- Приложение состоит из группы небольших, слабо связанных и легко изменяемых модулей
- Модули имеют хорошо документированный интерфейс взаимодействия
- Модули разрабатываются независимо
- Команда разработки модуля отвечает за весь жизненный цикл модуля

● Микросервисная архитектура

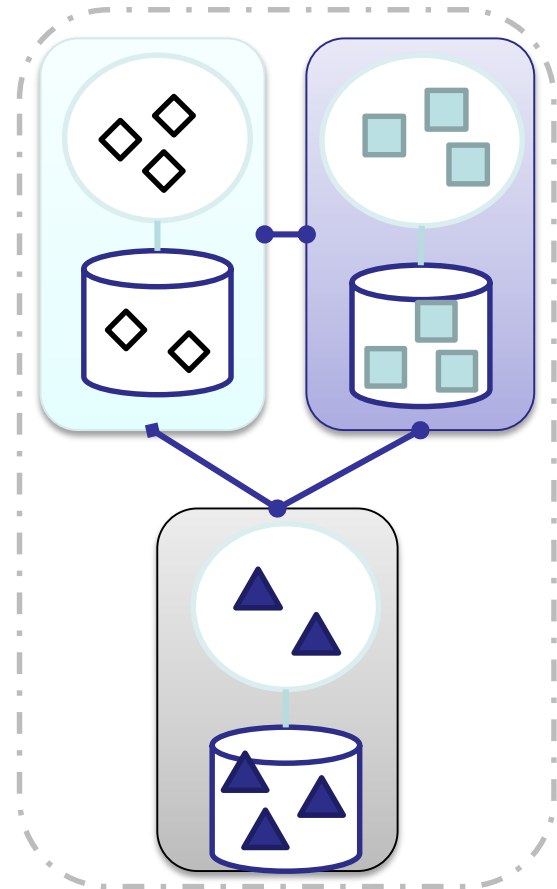
- Ускоряет разработку за счет уменьшения взаимодействия и координации между командами
- Уменьшает риск обновления за счет ограничения области изменения

Эволюция архитектуры приложения

Монолитное приложение



Микросервисная архитектура

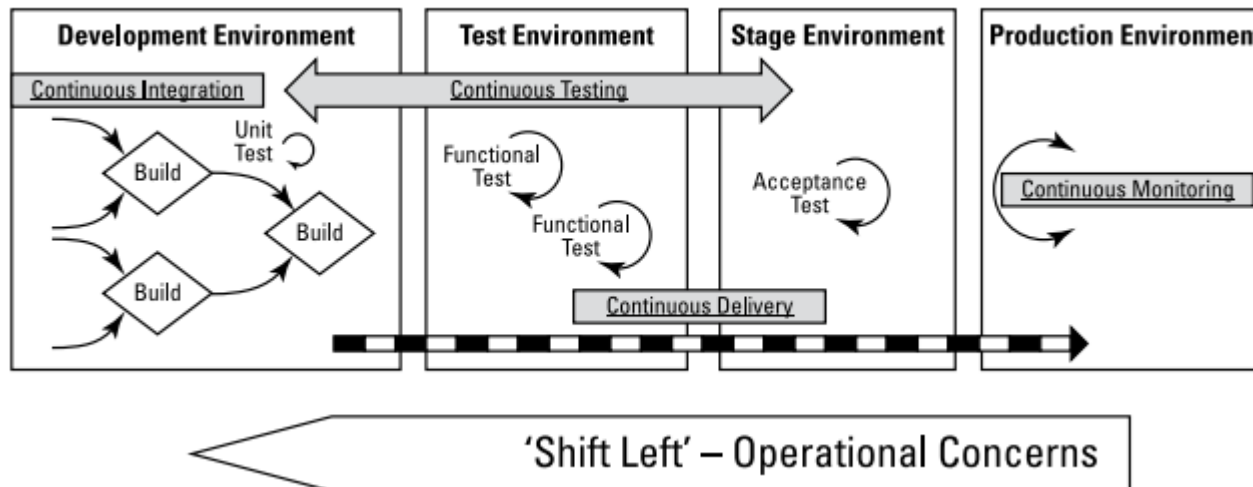


Новые задачи разработчиков

- Команда разработки микросервиса отвечает за весь жизненный цикл модуля
 - Разработка
 - Внутреннее тестирование
 - Интеграционное тестирование
 - Внедрение
 - Мониторинг и обратная связь
- Необходим инструментарий для:
 - Быстрого и автоматического внедрения
 - Внедрение новых версий в продуктивную среду
 - Изоляция новых версий
 - Перенос трафика на новые версии приложения
 - Постоянного мониторинга

Концепция DevOps

- Концепция DevOps предполагает тесное взаимодействие между разработчиками, тестировщиками и администраторами
- Цель: обеспечить эффективное создание и непрерывное обновление программ и услуг, уменьшить время Time-to-Market



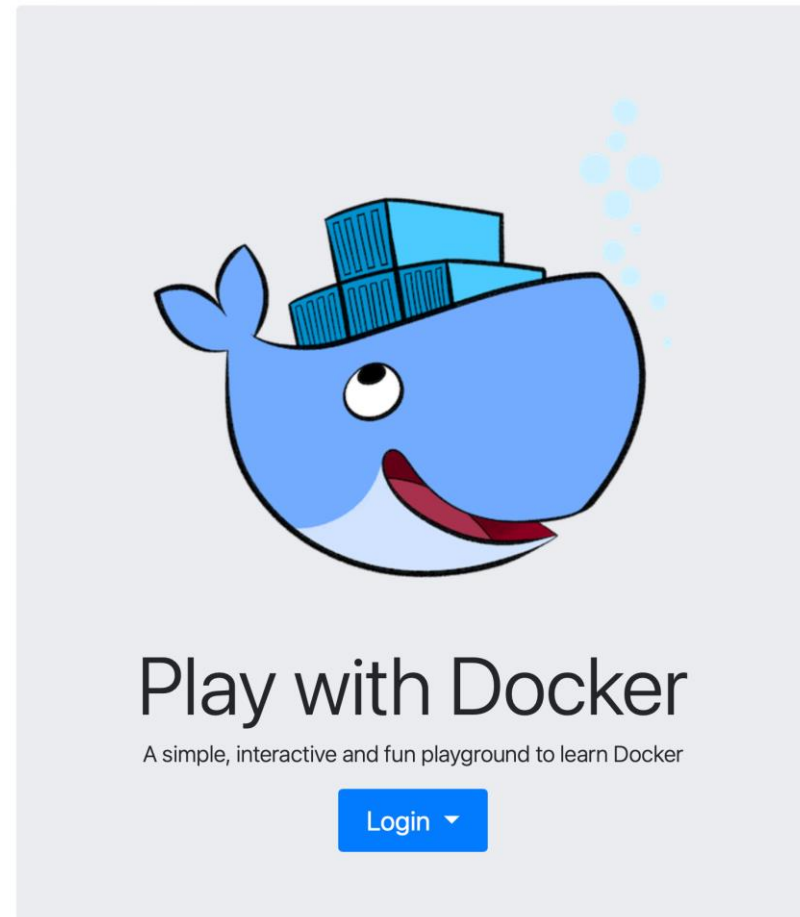
Docker Swarm

- Docker Swarm – кластерный режим работы Docker
 - Встроенный функционал, не требуется установка
 - Децентрализованная архитектура
 - Любой узел может выполнять управляющую роль
 - Декларативный подход к конфигурации
 - Масштабирование
 - Автоматический перезапуск сервисов
 - Обновление без простоя (Rolling Update)
 - Обнаружение имен сервисов и балансировка нагрузки

Установка Docker Swarm (1 из 2)

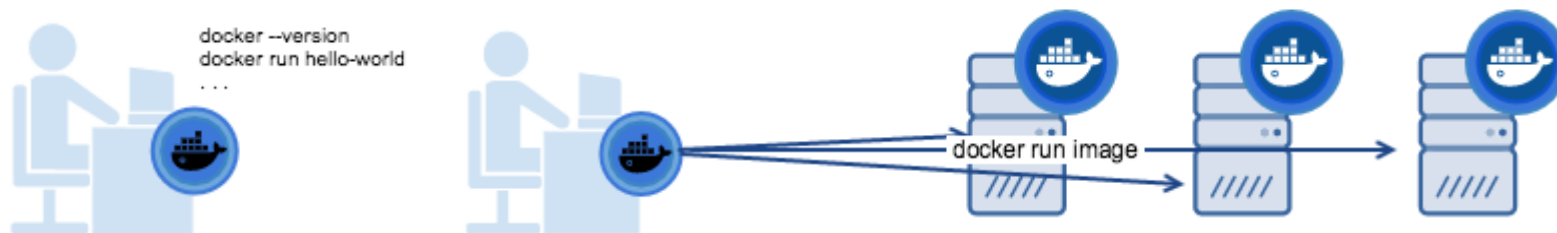
🔗 labs.play-with-docker.com

- Достаточно браузера
- Не требует оплаты, только аккаунт на docker.com
- Машины удаляются автоматически через 4 часа



Установка Docker Swarm (1 из 2)

- Docker-machine – инструмент для внедрения виртуальных машин с docker внутри
 - Локально с помощью VirtualBox
 - В облаке с помощью Digital Ocean, AWS и пр.



Docker Machine on Windows

```
docker-machine create --driver virtualbox my-manager  
docker-machine create --driver virtualbox my-worker1  
docker-machine create --driver virtualbox my-worker2
```

Инициализация кластера Swarm

Инициализация кластера Docker Swarm

- Создается сертификат безопасности
- Генерируется токен для авторизации рабочих узлов
- Генерируется токен для авторизации управляющих узлов
- Создается база данных Raft для хранения сертификатов, конфигурации и секретов
 - Хранится в зашифрованном виде на диске (с 1.13+)

```
$ docker swarm init
```

```
Swarm initialized: current node (v9o90dp3uyptpgpht) is now a manager.
```

To add a worker to this swarm, run the following command:

```
docker swarm join --token SWMTKN-5ngz12lp5o8sh3pg1bwmrtufnayt3cj  
71xyr3u3fy2xwnffil-562t77l2ipp8tyb24vlrs7bt2 192.168.16.129:2377
```

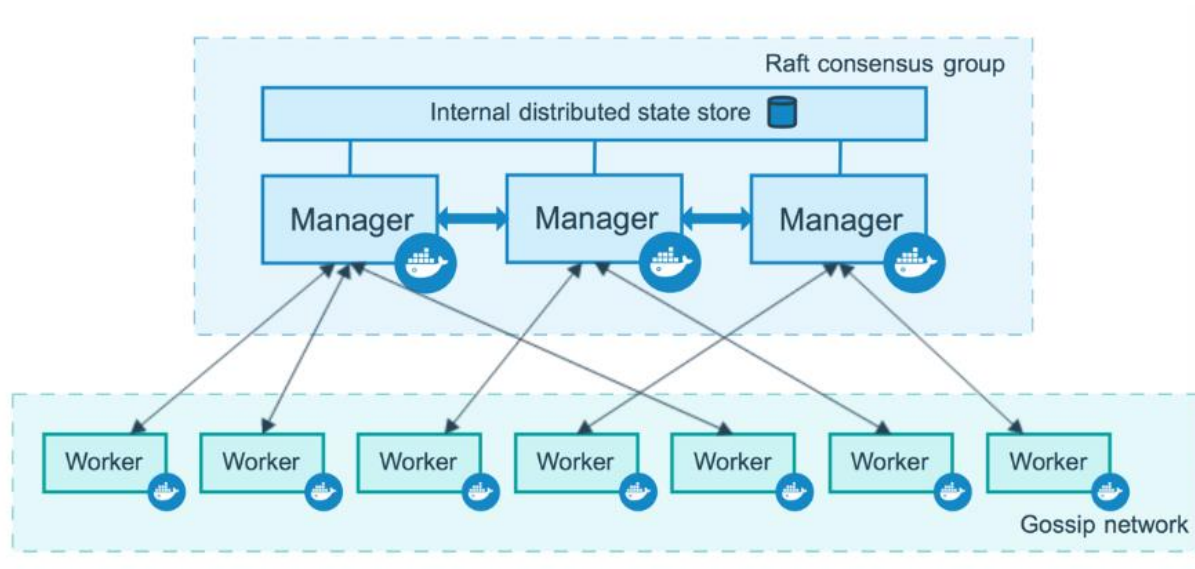
To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

Добавить управляющий узел

Добавить рабочий узел

Роли узлов в кластере Docker Swarm

- Узел – физическая, виртуальная машина или облачный ресурс с установленным Docker
 - Manager – управляет распределением задач в кластере
 - Группа управляющих узлов выбирают единого Leader
 - Worker – получает и исполняет задачи
 - Агент на рабочем узле отчитывается о состоянии выполняемых задач



Добавление узлов в кластер

Добавление рабочего узла

```
$ docker swarm join --token SWMTKN-5ngz12lp5o8sh3pg1bwmrtufnayt3cj  
71xyr3u3fy2xwnffil-562t77l2ipp8tyb24vlrs7bt2 192.168.16.129:2377
```

Добавление управляющего узла

```
docker swarm join-token manager
```

To add a manager to this swarm, run the following command:

```
docker swarm join \  
--token SWMTKN-1-61ztec5kyafptydic6jfc1i36k7kby-5vy9t8u35tuqm7vh67lrz9xp6 \  
192.168.16.129:2377
```

```
$ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS
v9o90dp3uyptpgpht *	node1	Ready	Active	Leader
izyp2gbf7xrdz42g7	node2	Ready	Active	
js24jmkp2o5zt5qps	node3	Ready	Active	Reachable

Текущий узел

Управляющие узлы

Управление узлами

- Минимум один управляющий узел в кластере
 - Рекомендовано 3-5 управляющих узлов
 - Обязательно нечетное количество
 - Кластер работает, пока активны $(N - 1) / 2$ управляющих узлов
- Возможна смена ролей с помощью `docker node promote` и `docker node demote`

```
$ docker node promote node3  
Node node-3 promoted to a manager in the swarm.
```

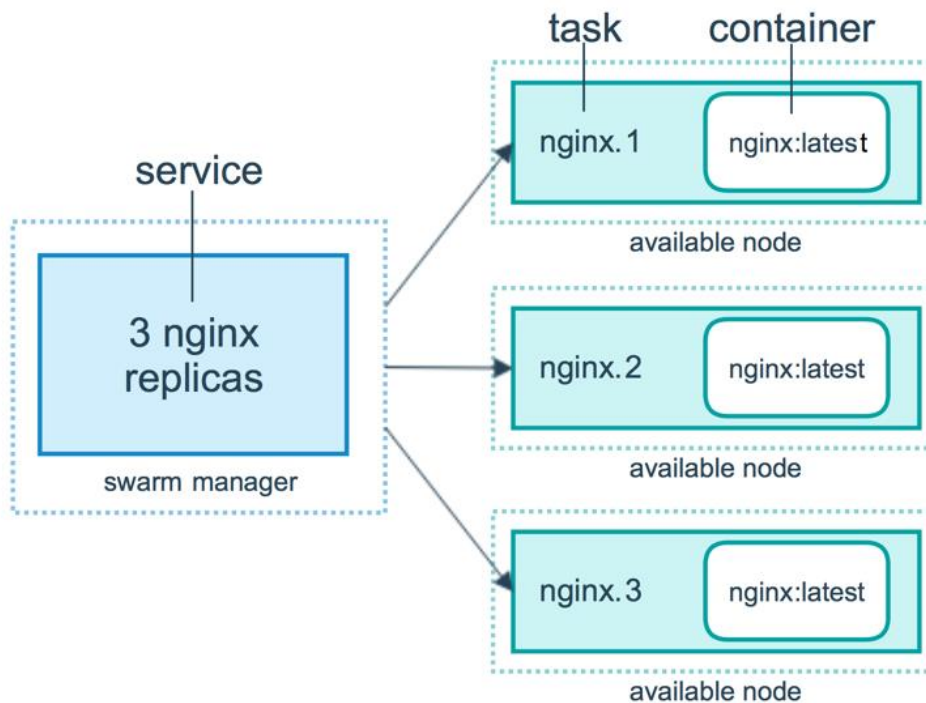
- Доступность узла для запуска сервисов
 - Перевести узел в режим обслуживания
 - Освободить ресурсы управляющего узла

```
$ docker node update --availability drain node1
```

Сервисы Docker Swarm

Сервис – способ внедрения приложений в кластере

- Образ контейнера и контекст работы
 - Порт для доступа внешних клиентов
 - Сеть для взаимодействия с сервисами внутри кластера
 - Ограничения CPU и памяти
 - Политика обновления
 - Количество реплик



Запуск сервиса (1 из 2)

Внедрение сервиса

```
$ docker service create --name test busybox ping 8.8.8.8
```

Просмотр развернутых сервисов

```
$ docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
kxzfkszyw61x	test	replicated	1/1	busybox:latest	

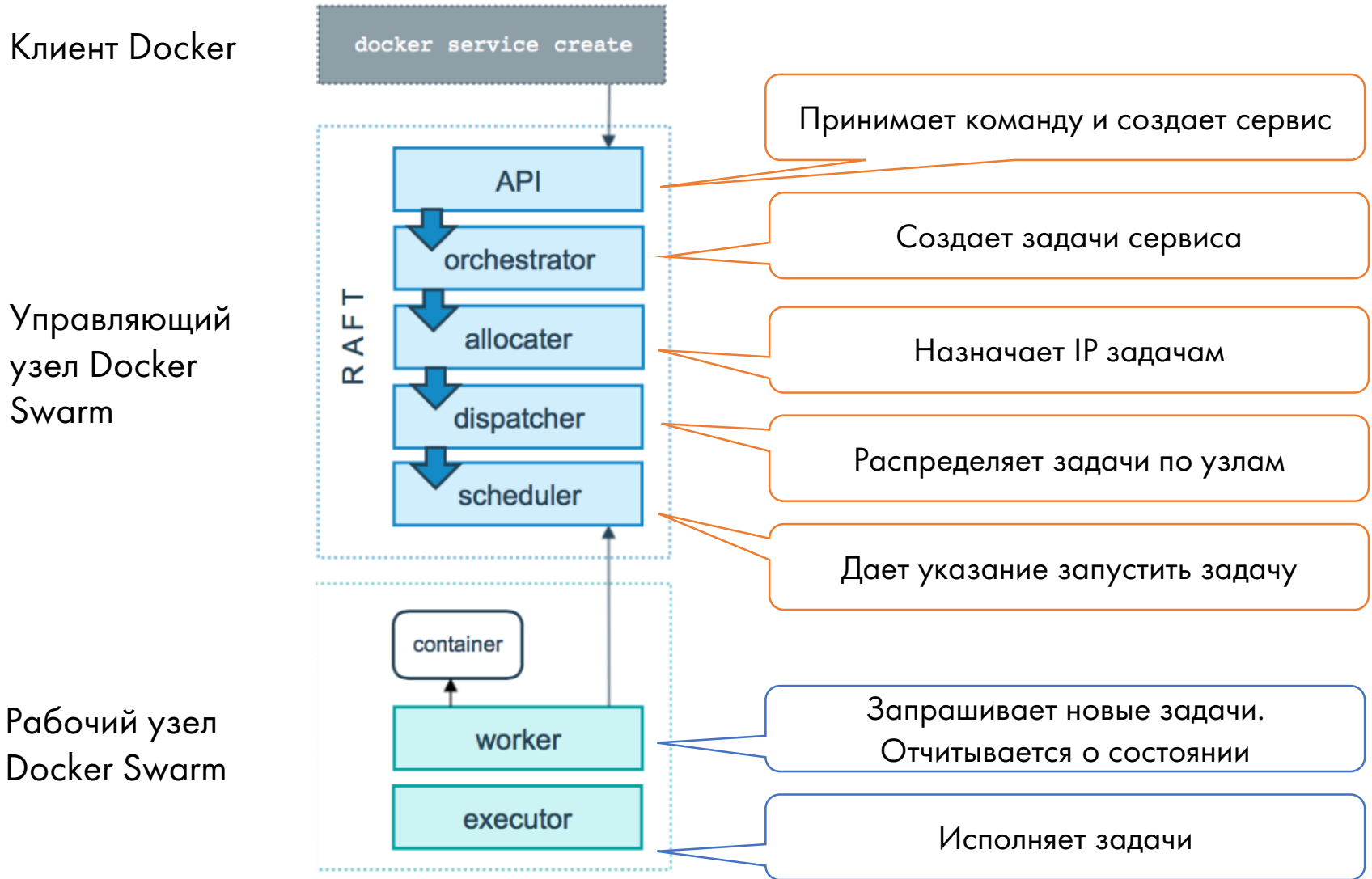
Просмотр состояния задач сервиса

- При остановке контейнера сервис автоматически перезапускает его

```
$ docker service ps test
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE
z0hjtsbmr15n	test.1	busybox:latest	node1	Ready	Ready 3 seconds ago
f431dlw1obp9	_ test.1	busybox:latest	node1	Shutdown	
Failed 3 seconds ago					

Запуск сервиса (2 из 2)



Просмотр свойств сервиса

```
$ docker service inspect --pretty test
```

```
ID: wvebyowj71zh38ov8rntyzqkg
```

```
Name: test
```

```
Service Mode: Replicated
```

```
Replicas: 1
```

```
Placement:
```

```
UpdateConfig:
```

```
Parallelism: 1
```

```
On failure: pause
```

```
Monitoring Period: 5s
```

```
Max failure ratio: 0
```

```
Update order: stop-first
```

```
RollbackConfig:
```

```
Parallelism: 1
```

```
On failure: pause
```

```
Monitoring Period: 5s
```

```
Max failure ratio: 0
```

```
Rollback order: stop-first
```

```
ContainerSpec:
```

```
Image: busybox:latest
```

```
Args: ping 8.8.8.8
```

```
Init: false
```

```
Endpoint Mode: vip
```

Формат отображения
удобный для чтения

Количество копий
контейнера

Политики обновления

Информация о запущенной
задаче

Политика балансировки
траффика

Масштабирование сервиса

Масштабирование сервиса (способ 1)

```
$ docker service scale test=3
test scaled to 3
overall progress: 1 out of 3 tasks
1/3: running
2/3: pending
3/3: new
```

Масштабирование сервиса (способ 2)

```
$ docker service update test --replicas 3
```

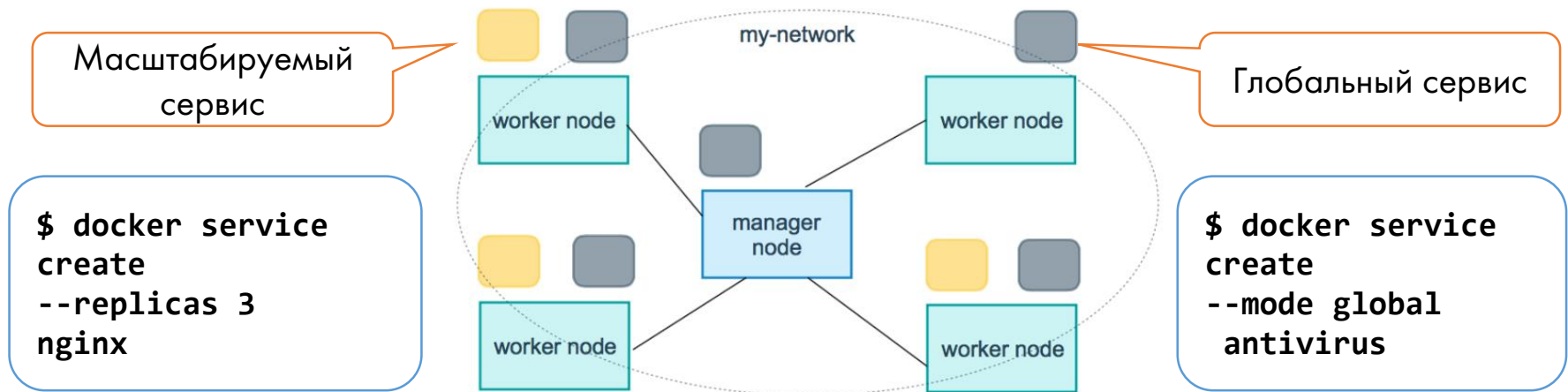
Просмотр реплик

```
$ docker service ps test
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE
z0hjtsbmr15n	test.1	busybox:latest	node1	Ready	Ready 45 seconds ago
f431dlw1obp9	_ test.1	busybox:latest	node1	Shutdown	Failed 56 seconds ago
qbjps3pl6mjr	test.2	busybox:latest	node2	Ready	Ready 4 seconds ago
jkvzaguo8g0r	test.3	busybox:latest	node3	Ready	Ready 4 seconds ago

Масштабируемые и глобальные сервисы

- Масштабируемый сервис (по умолчанию)
 - Желаемое количество реплик
 - Могут быть произвольно распределены по узлам
 - Можно ограничить количество реплик на узле
 - Пример: веб-сервер
- Глобальный сервис – по одной реплике на каждом узле
 - Пример: агент мониторинга, сканер антивируса и пр.



Обновление сервиса (1 из 2)

● Обновление сервиса – перезапуск задач с новыми параметрами

- Опции при обновлении те же, что и при создании

```
$ docker service update --limit-cpu 2 redis
```

● Параметры обновления

- Количество задач, обновляемых одновременно
- Задержка между обновлениями задач
- Длительность мониторинга после запуска
- Действия при ошибке (pause, continue, rollback) и пр.

```
$ docker service create --replicas 10 --name my_web --update-delay 10s \  
--update-parallelism 2 --update-failure-action continue -p 8080:80 nginx
```

Обновление сервиса (2 из 2)

Обновление образа контейнера

```
$ docker service update --image nginx:1.9.1 my_web
myweb
overall progress: 1 out of 5 tasks
1/5: running [=====>]
2/5: running [=====>]
3/5: preparing [=====>]
4/5: preparing [=====>]
5/5:
```

Одновременно обновляется не более 2 копий контейнера

Обновление портов контейнера

- Необходимо удаление и добавление порта

```
docker service update --publish-rm 8080 --publish-add 9090:80 my_web
```

Откат к предыдущей версии

```
$ docker service rollback my_web
```

Удаление сервиса



Удаление сервиса

```
$ docker service rm test
```

```
docker service inspect test  
[]  
Error: no such service: test
```

○ Удаление контейнеров может занять некоторое время

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	... NAMES
2bca37392557	busybox:latest	"ping 8.8.8.8"...	test.1.mjexnfbpxwjg72y9rb01ysk

```
$ docker ps
```

Кластерные сети. Overlay

- Внутренняя сеть, объединяющая машины кластера
 - Позволяет контейнерам взаимодействовать, даже если они размещены на разных узлах
 - Опционально AES-шифрование передаваемых данных
 - Управляющие данные шифруются по умолчанию
- Сервис может быть подключен к нескольким сетям

```
$ docker network ls
```

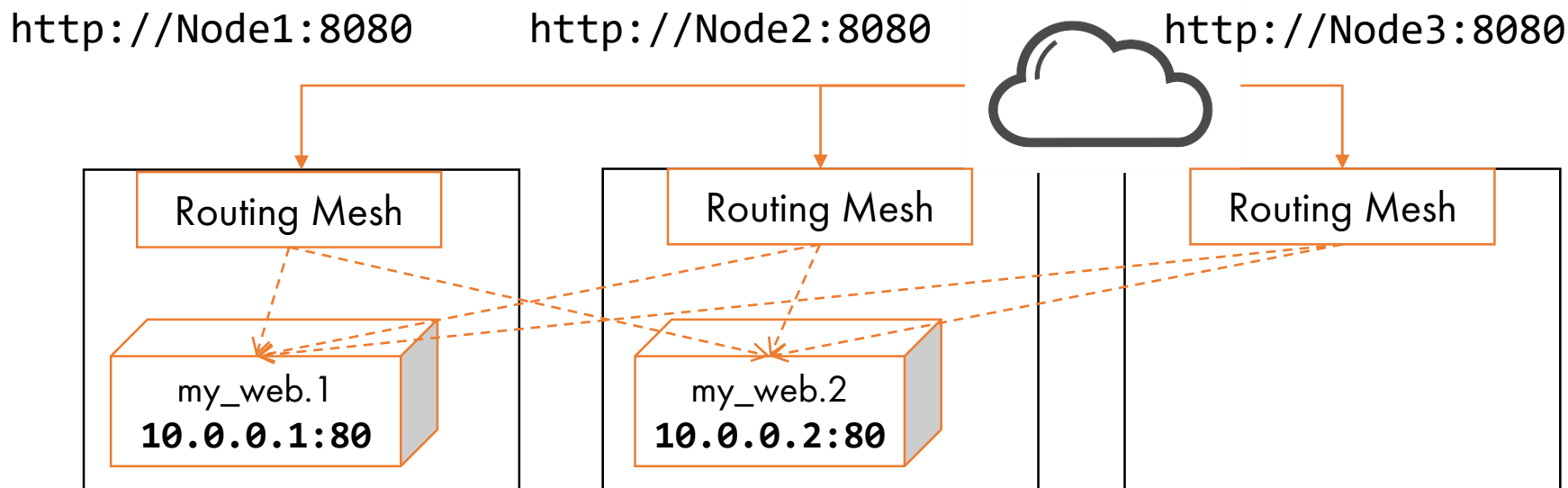
402b1b98d01e	bridge	bridge	local
e7e777825b74	docker_gwbridge	bridge	local
31901c60ecce	host	host	local
w7p74865py10	ingress	overlay	swarm
4960ab3bd366	none	null	local

Сеть, объединяющая демоны Docker в кластере

Сеть по умолчанию для контейнеров в кластере Docker Swarm

Распределение запросов. Routing Mesh

- Распределение запросов производится с помощью модуля Linux IPVS (IP Virtual Server)
 - Не требует дополнительной настройки
- Балансировка уровня L4 (на уровне портов TCP/UDP)
- Балансировка без учета состояния
 - Данные одного клиента могут быть направлены на разные реплики сервиса



Подключение сервиса к сети

Создание сети

```
$ docker network create --driver overlay my-network
```

Подключение сервиса к сети и публикация портов

```
$ docker service create --replicas=2 --network my-network -p 8080:80 nginx
```

Подключение клиента

- Возможно подключение через адрес любого узла кластера
- Распределение запросов по репликам сервиса происходит автоматически с помощью Routing Mesh

```
$ curl http://node1:8080  
Hi from instance 1  
$ curl http://node2:8080  
Hi from instance 2  
$ curl http://node3:8080  
Hi from instance 1
```

Распределение запросов – Routing Mesh

Распределение запросов. Альтернативы Routing Mesh

- Routing Mesh - единый адрес и порт для всех реплик сервиса

```
$ docker service create --replicas 3 --endpoint vip web-server
```

- DNS Round Robin позволяет обращаться к каждой реплике отдельно и балансировать нагрузку между репликами
 - Проблема: Routing Mesh не позволяет указать, какая реплика должна получить запрос
 - Сценарий: глобальный сервис – агенты мониторинга

```
$ docker service create --mode global --endpoint dnsrr monitoring-agent
```

- HAProxy, NGINX позволяют изолировать кластер от внешних клиентов с помощью прокси
 - Поддерживаются конфигурации как с использованием Routing Mesh (virtual ip), так и DNS Round Robin

- Основная задача – оркестровка контейнеров
 - развертывание – специфические средства для описания сложного приложения, состоящего из нескольких контейнеров, и его атомарного развертывания
 - отказоустойчивость/распределение нагрузки
 - приложения, требующие нескольких контейнеров, решающих разные задачи
 - масштабирование – возможность гибко увеличивать количество экземпляров запущенного приложения
 - управление – контроль за состоянием приложения, решение возникающих проблем
- Open source продукт

Задачи Kubernetes

- 🌀 **Kubernetes – оркестратор контейнеров**
 - Управление разворачиванием, размещением и жизненным циклом контейнеров
- 🌀 **Kubernetes позволяет**
 - Объединить группу узлов в кластер
 - Распределять контейнеры по узлам
 - Распределять запросы клиентов к контейнерам
 - Обеспечивать взаимодействие контейнеров
 - Отслеживать и перезапускать неработающие контейнеры
 - Масштабировать задачи для удовлетворения возросшей нагрузки
- 🌀 **Kubernetes поддерживает различные архитектуры от одного узла до облачных решений**
 - Может быть расширен с помощью плагинов, сетей и томов

Ключевые функции K8S

- Средства для выкладки/отката приложений
- Организация «единой точки входа» для несколько масштабируемых контейнеров
- Подключение провайдеров для персистентного хранения данных
- Управление конфигурацией и секретами
- Автоматизированное горизонтальное масштабирование
- Возможности для самовосстановления

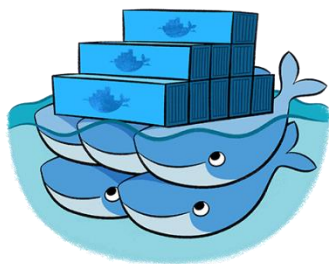
Что НЕ делает K8S

- ❯ CI/CD конвейеры
- ❯ Middleware услуги: серверы приложений, системы асинхронной обработки сообщений, шины, СУБД, хранилища данных
- ❯ Нет четкого регламента с точки зрения используемых языков, фреймворков, систем конфигурации и т.п.

Выбор оркестратора

🌀 Docker Swarm

- Простая установка
 - Встроен в Docker
- Простое управление
- Поставляется и поддерживается Docker
- Более простая отладка
- Поддерживает многие сценарии



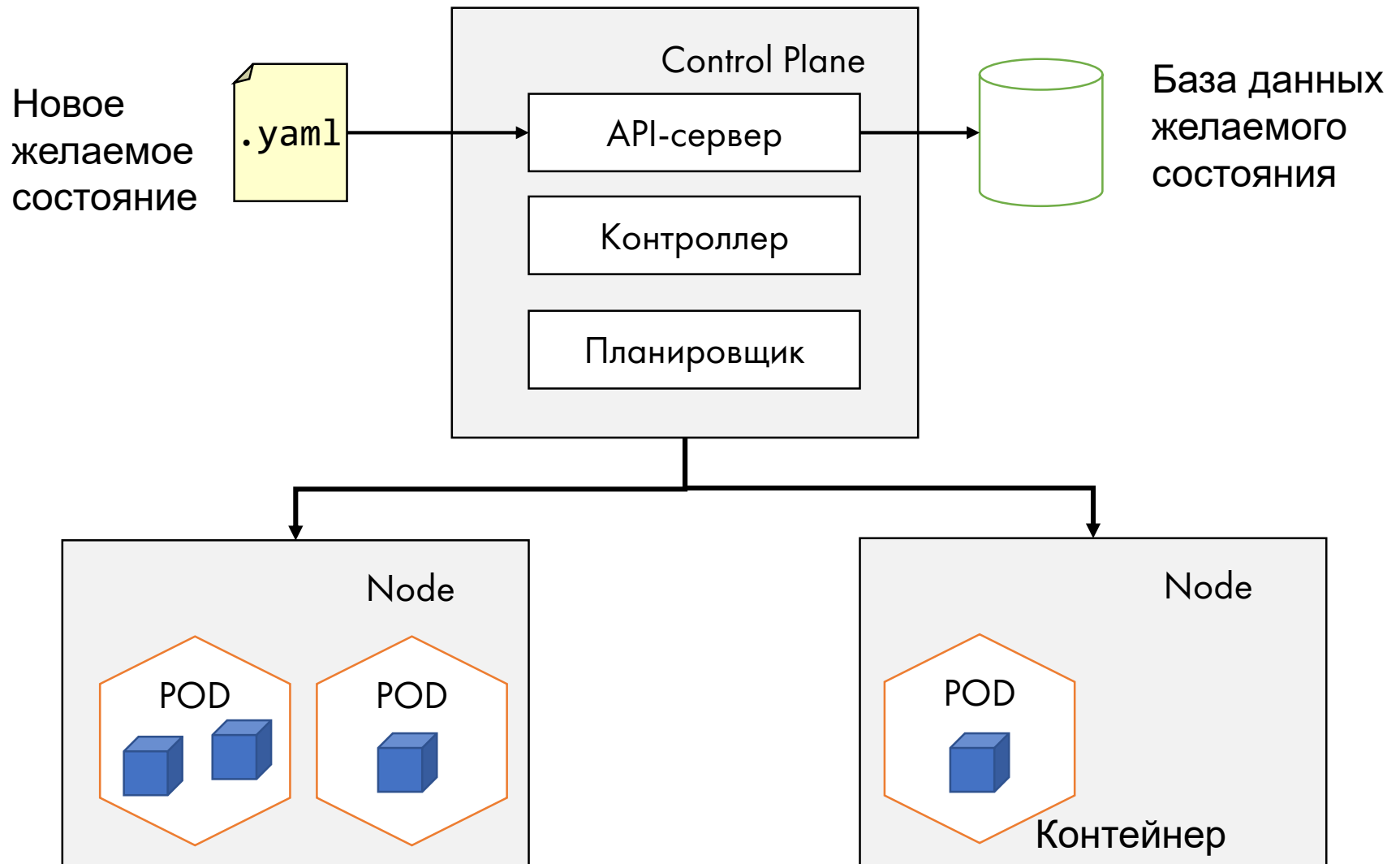
🌀 Kubernetes

- Более сложная установка и управление
- Встроен и поддерживается большинством облачных провайдеров
- Больше возможностей настройки
- Поддерживает большинство сценариев

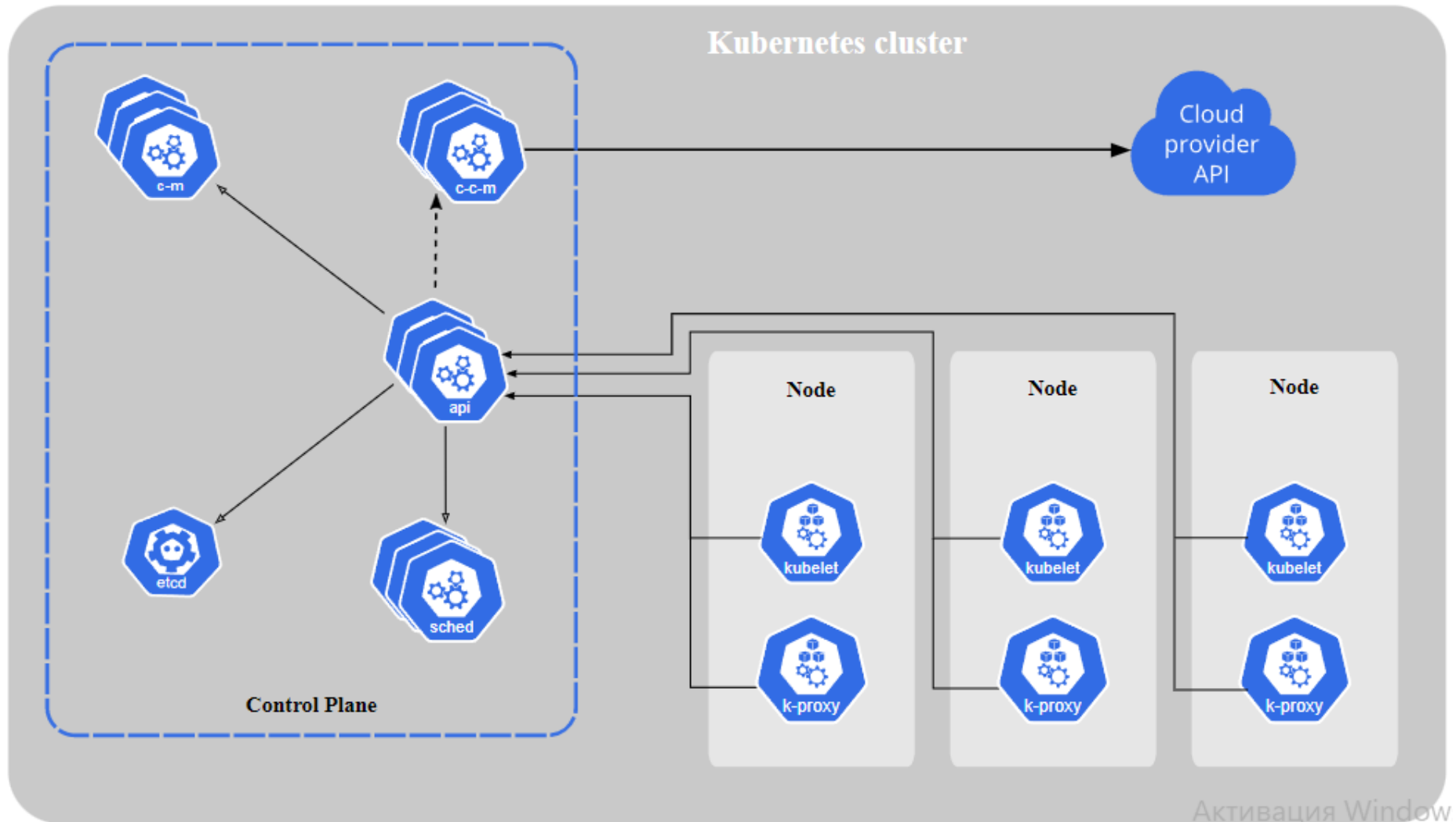
VS



Архитектура кластера Kubernetes



Архитектура кластера Kubernetes



Желаемое и текущее состояние

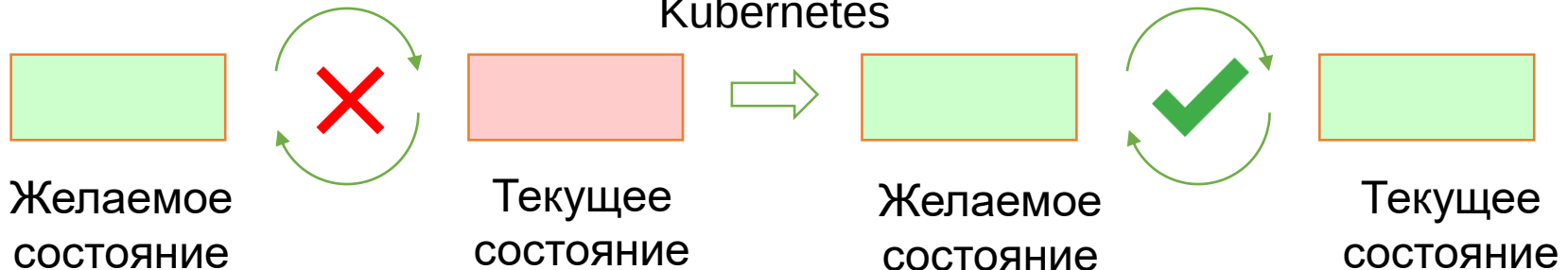
● Состояние – ключевое понятие Kubernetes

- Желаемое состояние
- Текущее состояние

● Декларативная модель управления

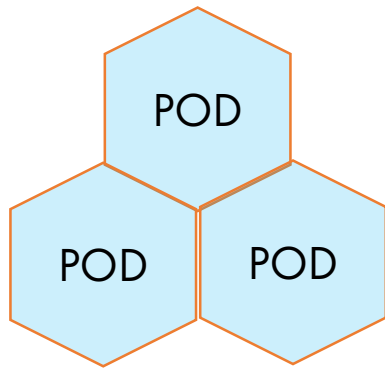
- Пользователь описывает желаемое состояние в конфигурационных файлах .yaml и передает их API-серверу
- Контроллер анализирует текущее состояние объектов кластера и формирует команды для запуска/остановки/изменения контейнеров

● Цель Kubernetes – привести текущее состояние к желаемому



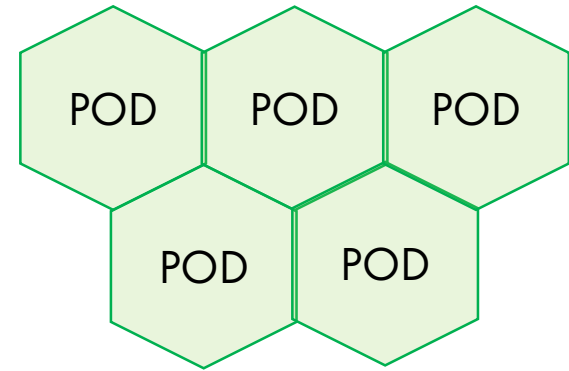
Императивное и декларативное управление

Императивное управление



Добавить 2 POD

Обновить версию
образа

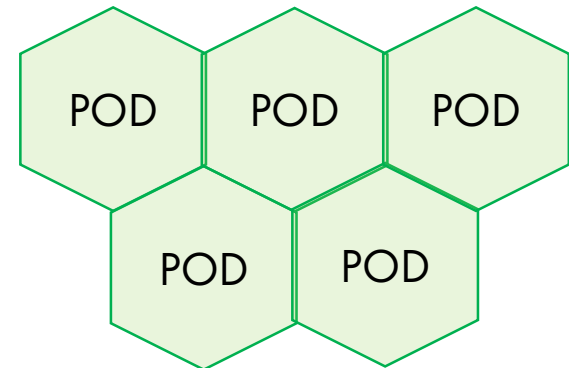


Декларативное управление



Необходимо:

- 5 POD
- Версия образа 2.0



Выбор стратегии управления

● Императивное управление

- Более простой подход на начальном этапе
 - Обучение или управление небольшим кластером
- Усложняется автоматизация
 - Различные команды и опции для разных объектов
- Нет контроля изменений

● Декларативный подход

- Контроль изменений YAML-файлов
 - Например, история изменений в GitHub
- Широкие возможности автоматизации
- Рекомендованный подход для продуктивных окружений

● ВАЖНО: не смешивать подходы управления одним кластером

kubectl – введение

- Утилита, позволяющая работать с кластером, как в императивном, так и в декларативном ключе
- Есть отдельный сайт с документацией
 - <https://kubectl.docs.kubernetes.io/>

kubectl – типы задач

Тип задач	Область применения	Комментарий
Декларативное управление ресурсами (объектами)	Развертывание приложений и управление ими	
Императивное управление ресурсами (объектами)	Быстрая работа в среде для разработки	Создание объектов без предварительной подготовки <code>yaml</code>
Вывод статусной информации	Отладка	Получение информации о запущенных приложениях
Взаимодействие с контейнерами	Отладка	Копирование, запуск процесса, логи
Управление кластером	Административные задачи	Управление узлами кластера

kubectl – декларативное управление

```
# vim deployment.yaml
# kubectl apply -f deployment.yaml
#
# vim secret.yaml
# kubectl apply -f secret.yaml
#
# vim pvc.yaml
# kubectl apply -f pvc.yaml
```

kubectl – императивное управление

```
# kubectl run nginx --image=nginx --port=80
#
# kubectl expose rc nginx --type=LoadBalancer --name nginx
#
# kubectl create secret generic db-user-pass \
  --from-literal=username=devuser \
  --from-literal=password='S!B\*d$zDsb='
#
# kubectl scale rc nginx --replicas=5
```


YAML – Yet Another Markup Language

- Основной язык для написания манифестов в K8S
- Структурированный язык, более лаконичный и человекочитаемый, чем XML и JSON (хотя последний тоже может использоваться в K8S)
- Применяется в других решениях, например Ansible
- Пробелы (отступы) – важная часть структуры документа

Объекты кластера Kubernetes

Pod – базовый исполняемый объект

- Несколько контейнеров
- Тома
 - Все контейнеры имеют доступ к томам Pod
- Сетевой доступ
 - Все контейнеры используют общий IP-адрес и общий набор портов

Service – сетевой объект

- Pod может быть остановлен и запущен повторно
 - При этом может получить новый IP-адрес
 - Может быть увеличено количество реплик Pod
- Service определяет логические группы Pod и политику сетевого доступа

Pod. Описание

my-pod.yml

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
```

```
  containers:
  - name: web-container
    image: nginx:1.17
    command:
    - 'nginx'
    - '-g'
    - 'daemon off;'
    ports:
    - containerPort: 80
```

Тип объекта

Контейнеры внутри Pod

Образ из репозитория
NB: K8s не поддерживает сборку образа

Порты, публикуемые
контейнером

Pod. Создание и просмотр (1 из 2)

```
$ kubectl create -f my-pod.yml  
pod/nginx-pod created
```

Наблюдать за
изменениями состояния

```
$ kubectl get pods --watch
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-pod	0/1	Pending	0	1s
nginx-pod	0/1	ContainerCreating	0	0m30s
nginx-pod	1/1	Running	0	1m10s
nginx-pod	1/1	Error	0	21m
nginx-pod	1/1	Running	0	21m15s

```
$ kubectl describe pod/nginx-pod
```

```
Name:          nginx-pod  
Namespace:     default  
Node:          minikube/172.17.0.26  
Labels:        app=nginx  
Annotations:   <none>  
Status:        Running  
IP:            172.18.0.4  
...
```

Pod. Создание и просмотр (2 из 2)

```
$ kubectl describe pod/nginx-pod
```

```
...
```

```
Containers:
```

```
  web-container:
```

```
    Container ID:  docker://9f6f13a4a51a2f0f67ed2303dde4fa0f2943442e578
```

```
    Image:         nginx:1.17
```

```
    Port:         80/TCP
```

```
    Host Port:    0/TCP
```

```
    Command:
```

```
      nginx
```

```
      -g
```

```
      daemon off;
```

```
    Mounts:
```

```
      /var/run/secrets/kubernetes.io/serviceaccount (ro)
```

```
...
```

```
Events:
```

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	Scheduled	3m18s	default-scheduler	Successfully assigned default/nginx-pod to minikube
Normal	Pulled	2m14s (x2 over 3m17s)	kubelet, minikube	Container image "nginx:1.17" already present on machine
Normal	Created	2m14s (x2 over 3m17s)	kubelet, minikube	Created container web-container

Service. Описание

my-service.yml

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  type: NodePort
  ports:
    - name: http
      protocol: TCP
      port: 8080
      targetPort: 80
```

Вид описания

Selector определяет группу объектов, для которых применяется сетевая спецификация, соответствует **label** в описании Pod

Kube-proxy будет перенаправлять трафик, поступающий на порт узла 8080, в контейнеры, описанные полем **selector**, на порт 80

Создание Service

```
$ kubectl create -f my-service.yml
service "nginx-service" created
```

```
$ kubectl get svc nginx-service
```

NAME	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
my-service	100.70.40.2	<none>	8080:31882/TCP,3000:32562/TCP	8s

```
$ kubectl describe svc nginx-service
```

Name: nginx-service
Namespace: default
Labels: <none>
Selector: app=nginx
Type: NodePort
IP: 10.107.131.100
Port: http 9376/TCP
TargetPort: 80/TCP
NodePort: http 31882/TCP
Endpoints: 192.168.219.70:80
Session Affinity: None
External Traffic Policy: Cluster
Events: <none>

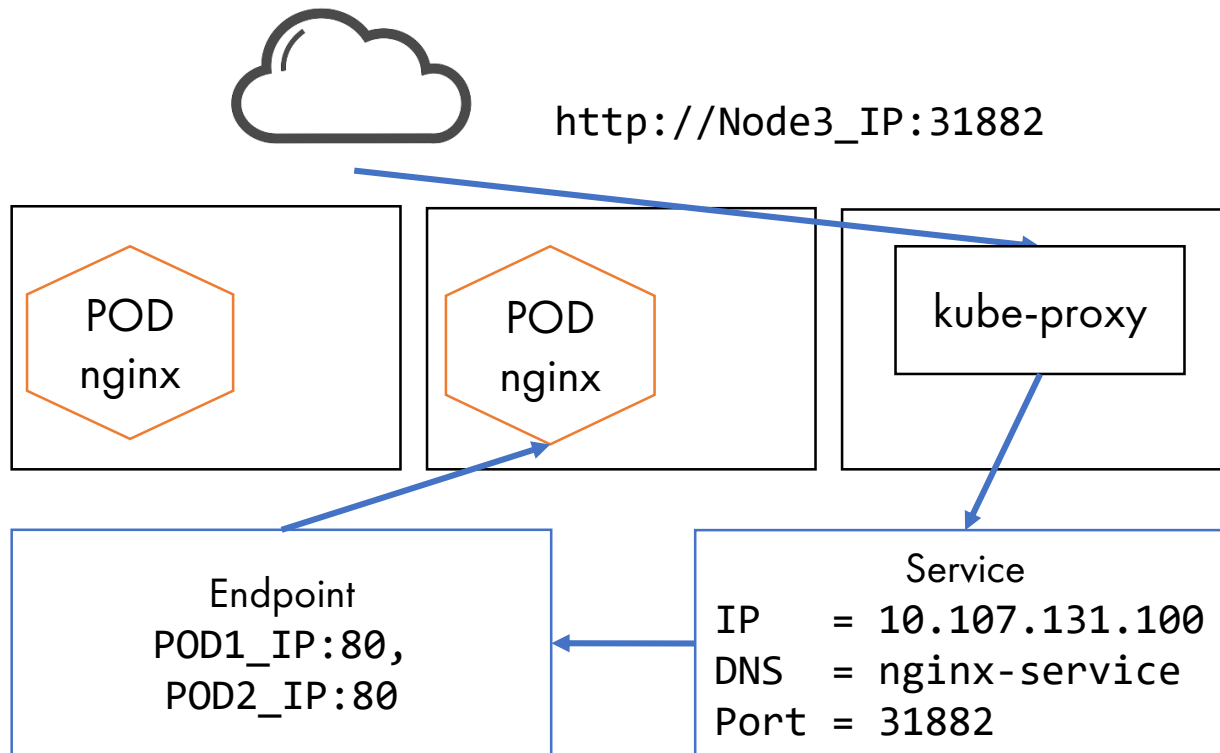
Внутреннее подключение
`http://10.107.131.100:8080`

Внешнее подключение
`http://NodeIP:31882`

Service. Принцип работы

Сервис добавляет уровень абстракции между клиентами и POD

- Стабильный IP
- Распределение запросов между репликами POD
- Связь сервиса и POD – с помощью меток и селекторов



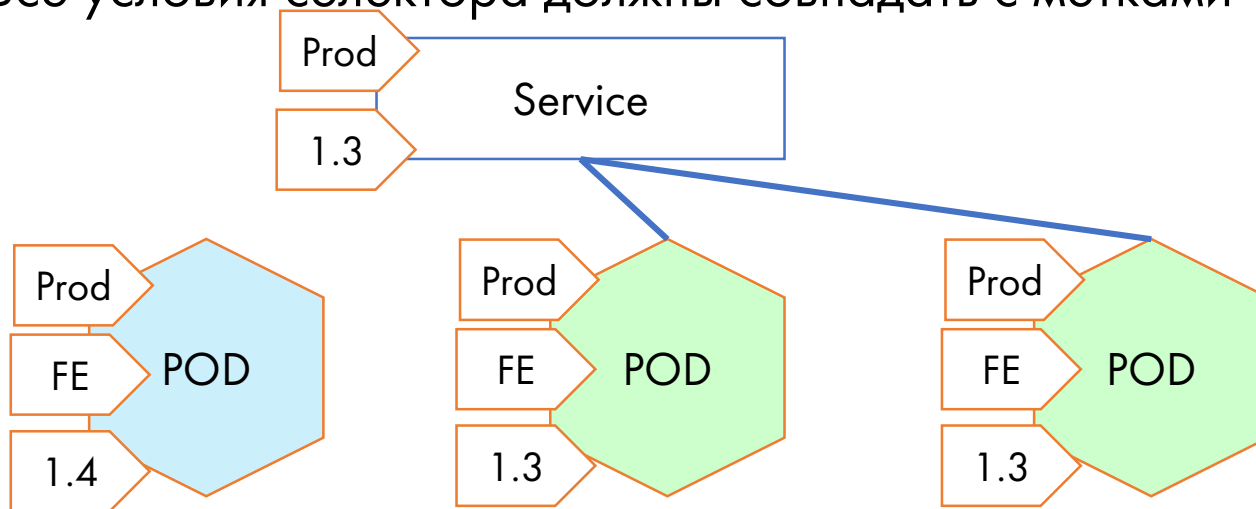
Метки и селекторы

🕒 **Метка – неуникальный идентификатор объекта Kubernetes внутри Namespace**

- Позволяет сопоставлять объекты Kubernetes
- Пара ключ – значение, например `app: nginx`

🕒 **Селектор выбирает объект, метки которого соответствуют условиям**

- Все условия селектора должны совпадать с метками



Идентификаторы объекта Kubernetes

Имя и UID (назначается кластером)

- Имя уникально в пределах Namespace

Namespace – виртуальный кластер

- Default – виртуальный кластер для объектов клиента
- Kube-system – виртуальный кластер для объектов K8s

```
$ kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS
calico-kube-controllers-78f8f67c4d-cjqm7	1/1	Running	0
calico-node-lz8bw	1/1	Running	8
coredns-fb8b8dccf-d57vd	1/1	Running	2
coredns-fb8b8dccf-h6v6q	1/1	Running	2
etcd-master	1/1	Running	0
kube-apiserver-master	1/1	Running	1
kube-controller-manager-master	1/1	Running	18
kube-proxy-wz75z	1/1	Running	0
kube-scheduler-master	1/1	Running	15

● ReplicaSet описывает желаемое количество запущенных Pod

- Для каждого типа Pod – свой набор реплик

my-rs.yml

```
apiVersion: v1
kind: ReplicaSet
metadata:
  name: frontend
spec:
```

```
  replicas: 3
  selector:
    matchLabels:
      tier: frontend
```

```
  template:
    metadata:
      labels:
        tier: frontend
    spec:
      containers:
        - name: web-server
          image: nginx
```

Количество запущенных
копий Pod

Применимо к Pod с
указанной меткой

Описание Pod

Deployment. Описание

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.17
          ports:
            - containerPort: 80
```

🔄 **Объект Deployment** позволяет обновлять сервисы и возвращаться к предыдущим версиям без простоя

🔄 **Deployment** включает в себя **ReplicaSet**

- Рекомендуется использовать **Deployment**

Описание набора реплик

Описание Pod

Работа с Deployment

🔄 Применение Deployment

```
$ kubectl apply -f nginx_depl.yml  
deployment.apps/nginx-deployment created
```

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-deployment-6dd86d77d-k4rdd	0/1	ContainerCreating	0	6s
nginx-deployment-6dd86d77d-ss46x	0/1	ContainerCreating	0	6s
nginx-deployment-6dd86d77d-zrk54	0/1	ContainerCreating	0	6s

🔄 Кластер всегда приводит объекты к желаемому состоянию

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-deployment-6dd86d77d-k4rdd	1/1	Running	0	46m
nginx-deployment-6dd86d77d-ss46x	1/1	Error	0	46m
nginx-deployment-6dd86d77d-zrk54	1/1	Running	0	46m



nginx-deployment-6dd86d77d-ss46x	1/1	Running	1	46m
----------------------------------	-----	----------------	----------	-----

Различие между Deployment и ReplicaSet

Deployment	ReplicaSet
Абстракции высокого уровня, управляющие наборами реплик. Предоставляет дополнительные функции, такие как поочередное обновление, откат и управление версиями приложения.	Абстракция низкого уровня, которая управляет желаемым количеством реплик пода. Обеспечивает базовые механизмы масштабирования и самовосстановления.
Управляет шаблоном подов и использует наборы реплик, чтобы гарантировать работу указанного количества реплик модуля.	ReplicaSet управляет только желаемым количеством реплик пода.
Обеспечивает механизм последовательного обновления и отката приложения, обеспечивая плавное обновление и сокращая время простоя.	Приложения необходимо вручную обновлять или откатывать.
Обеспечивает управление версиями приложения, позволяя управлять несколькими версиями одного и того же приложения. Это также позволяет легко вернуться к предыдущей версии в случае необходимости.	ReplicaSet не предоставляет эту возможность.

Удаление объекта Kubernetes

```
$ kubectl apply -f nginx_depl.yml  
deployment.apps/nginx-depl created
```

```
$ kubectl get deployment
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx-depl	2/2	2	2	2m44s

```
$ kubectl get replicaset
```

NAME	DESIRED	CURRENT	READY	AGE
nginx-depl-86566f869b	2	2	2	2m48s

```
$ kubectl get pods
```

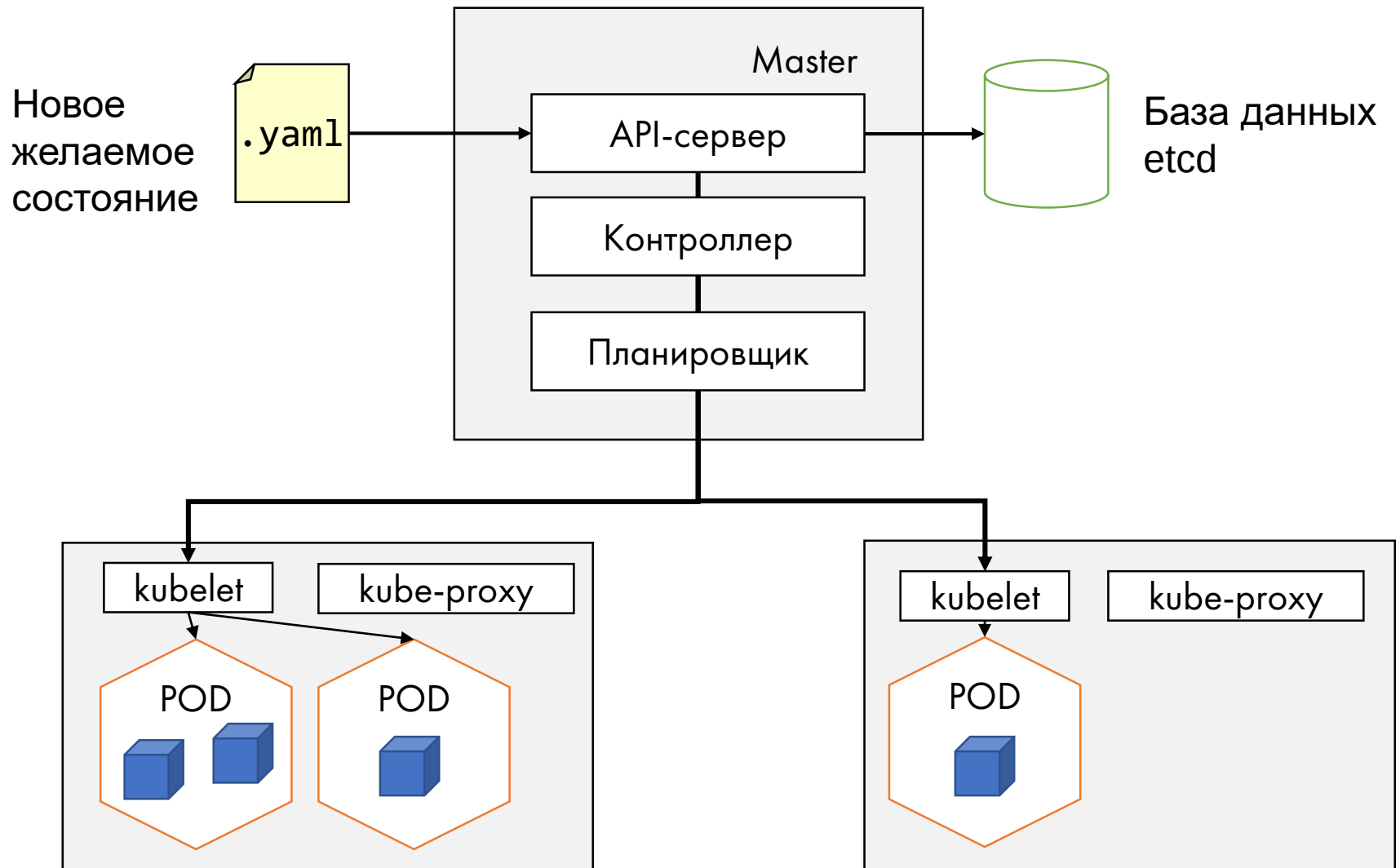
NAME	READY	STATUS	RESTARTS	AGE
nginx-depl-86566f869b-5wf4l	1/1	Running	0	2m50s
nginx-depl-86566f869b-9vpnj	1/1	Running	0	2m50s

```
$ kubectl delete deployment nginx-depl  
deployment.extensions "nginx-depl" deleted
```

```
$ kubectl get deployment  
$ kubectl get replicaset  
$ kubectl get pods
```

Удаление объекта в Kubernetes приводит к каскадному удалению созданных им объектов

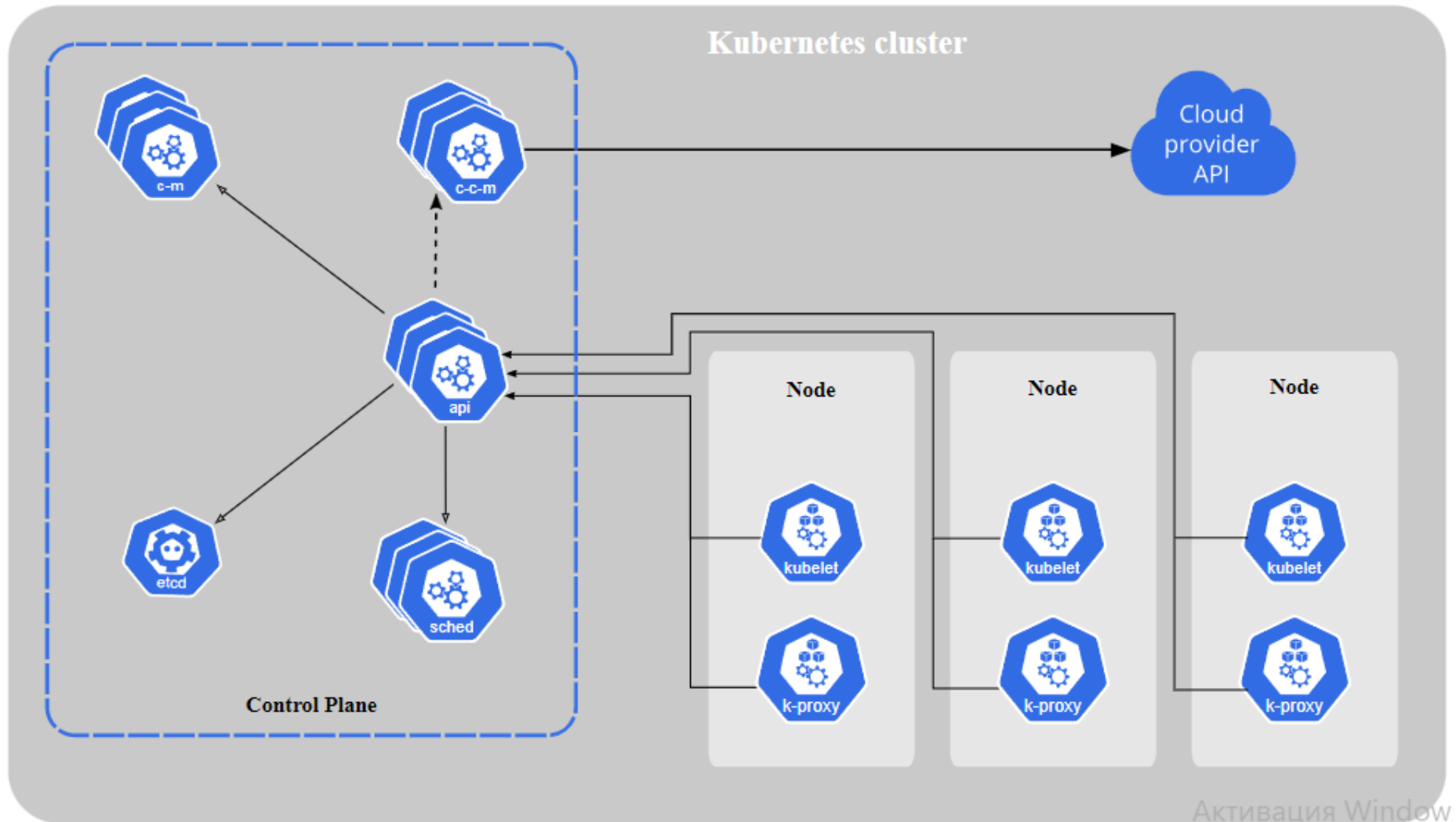
Архитектура кластера Kubernetes



Управляющий узел

- Все управляющие компоненты узлов – контейнеры
 - Теоретически могут работать на любой машине кластера
 - Как правило компоненты управления сосредоточены на одном или нескольких управляющих узлах
- Control Plane – основной управляющий узел кластера Kubernetes
 - Простейший кластер может иметь один узел Master
 - Поддерживаются конфигурации с двумя и более управляющими узлами
 - Высокая доступность
 - Для продуктивных окружений
 - Как правило, Master не используется для работы POD

Архитектура кластера Kubernetes



Компоненты управляющих (control plane) узлов

- kube-apiserver – API сервер, который принимает запросы для изменения/просмотра состояния кластера
- etcd – хранилище типа ключ-значение для персистентного сохранения всех кластерных данных
- kube-scheduler – выбирает узлы для запуска вновь запущенных подов (контейнеров)
- kube-controller-manager – процесс, отвечающий за работу контроллеров:
 - Node controller – следит за сбоями узлов кластера
 - Job controller – следит за объектами Job и соответствующими подами
 - Endpoints controller – заполняет свойство Endpoints в кластерных объектах, то есть, обеспечивает связность сервисов и подов
 - Service Account & Token controllers – создает новые учетные записи и токены для новых пространств имен (namespaces)

Компоненты обычных узлов

- kubelet – обеспечивает возможность запуска контейнеров в подах
- kube-proxy – сетевая прокси-служба, которая делает возможным работу объектов типа сервис (service)
- Container runtime – движок контейнерной виртуализации (по умолчанию используется CRI-O)

Дополнительные компоненты кластера

Кластерный DNS

- Обслуживает разрешение имен для сервисов кластера
- Контейнеры, запущенные в Kubernetes, автоматически подключаются к кластерному DNS

Веб-интерфейс

- Управление и отслеживание проблем кластера и приложений, запущенных в Kubernetes

Мониторинг ресурсов контейнера

- Сохраняет метрики данных контейнеров в базе данных и предоставляет интерфейс для доступа к данным

Журналирование

- Сохраняет логи отдельных контейнеров в центральное хранилище, предоставляет интерфейс для доступа

Итоги

В данной лекции были рассмотрены следующие темы:

 Задачи, требующие оркестровки приложений и кластеризации

 Docker Swarm

- ⦿ Создание кластера
- ⦿ Узлы в кластере
- ⦿ Сервисы
- ⦿ Масштабирование сервисов
- ⦿ Распределение нагрузки

 Kubernetes

- ⦿ Задачи, архитектура и основные сущности
- ⦿ Декларативный и императивные подходы в управлении
- ⦿ Сервисы
- ⦿ Deployment и ReplicaSet
- ⦿ Компоненты Kubernetes